

On-Prem Kubernetes Upgrade Runbook (kubeadm)

Example target version: v1.37.0 · Substitute your actual current/target versions throughout

Upgrade one minor version at a time → control plane first (first node with "apply", rest with "node") → worker nodes last, one at a time

Golden rules: you can only upgrade **one minor version at a time** (e.g. 1.36 → 1.37, never 1.36 → 1.38 directly — skipping minors isn't supported). The control plane is always upgraded before any worker nodes. On a multi-control-plane cluster, only the *first* control plane node runs `kubeadm upgrade apply` — every other node (control plane or worker) uses `kubeadm upgrade node`.

Before you start

```
kubectll get nodes -o wide          # confirm current versions across all nodes
kubectll version
```

Back up etcd

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot-$(date +%F).db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

Check compatibility

- Review the target version's release notes for removed/deprecated APIs your manifests use (`kubectll convert` or a tool like Pluto can catch these).
- Confirm your CNI plugin and any add-ons (ingress controller, CSI drivers, etc.) support the target Kubernetes version.

Step 1 · Upgrade the first control plane node

Point the package repo at the target minor version — Kubernetes package repos are versioned per-minor since the migration off the old `apt.kubernetes.io`:

```
# Debian/Ubuntu – example target v1.37
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.37/deb/ ' \
| sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y --allow-change-held-packages kubeadm=1.37.0-*
kubeadm version
```

RHEL/CentOS: update the `baseurl` in `/etc/yum.repos.d/kubernetes.repo` to the `v1.37` path, then `dnf install -y kubeadm-1.37.0-* --disableexcludes=kubernetes`.

```
sudo kubeadm upgrade plan
```

Review the output carefully — it lists exactly which component versions will change and flags anything it can't upgrade automatically.

```
sudo kubeadm upgrade apply v1.37.0
```

Now drain and upgrade kubelet/kubectll on this same node:

```
kubectll drain <control-plane-node-1> --ignore-daemonsets

sudo apt-get install -y --allow-change-held-packages kubelet=1.37.0-* kubectll=1.37.0-*
sudo systemctl daemon-reload
sudo systemctl restart kubelet

kubectll uncordon <control-plane-node-1>
```

Step 2 · Upgrade the remaining control plane nodes (repeat per node)

```
# repo update + package install same as Step 1, then:
sudo kubeadm upgrade node

kubectll drain <control-plane-node-N> --ignore-daemonsets
sudo apt-get install -y --allow-change-held-packages kubelet=1.37.0-* kubectll=1.37.0-*
```

```
sudo systemctl daemon-reload && sudo systemctl restart kubelet
kubectl uncordon <control-plane-node-N>
```

Step 3 · Upgrade worker nodes, one at a time

Upgrade workers one at a time (or in small batches, respecting PodDisruptionBudgets). Drain from a machine with cluster-admin `kubectl` access:

```
kubectl drain <worker-node-1> --ignore-daemonsets --delete-emptydir-data
```

Then on the worker node itself:

```
# repo update to v1.37 + package install (kubeadm, kubelet, kubectl if present) same as Step 1
sudo kubeadm upgrade node
sudo systemctl daemon-reload && sudo systemctl restart kubelet
```

Back on the admin machine:

```
kubectl uncordon <worker-node-1>
```

Repeat for each remaining worker node.

Verify when done

```
kubectl get nodes -o wide      # every node should show the new VERSION
kubectl get pods -A           # confirm workloads rescheduled healthily
```

Things worth knowing

- **Version skew policy** — kubelets are allowed to lag a few minor versions behind the control plane (this window has been extended in recent releases), but best practice is to keep every node within one minor version of the control plane rather than relying on the outer bound of that skew. Don't let worker upgrades lag indefinitely.
- `kubectl version` should stay within one minor version (either direction) of the API server.
- **No skipping minor versions** — going from, say, 1.34 to 1.37 means walking through each minor version in sequence (1.34→1.35→1.36→1.37). There's no shortcut to jump three minors at once.
- **Only the first control plane node runs `kubeadm upgrade apply`** — running it on additional control plane nodes is incorrect; they use `kubeadm upgrade node` instead.
- **Always back up etcd before starting** — this is the step most likely to be skipped and most needed if something goes wrong.

Sources

- [Upgrading kubeadm clusters — Kubernetes documentation](#)
- [How to Upgrade Kubernetes Cluster Using Kubeadm? — DevOpsCube](#)
- [CKA Cluster Upgrade with kubeadm: Step-by-Step Guide — Sailor.sh](#)